Step 2 — A Real ReAct Agent That Calls a Discovered Tool — Solution

> Back to the step · Hint 1 · Hint 2

Story: this is the point of the whole project — an agent that acts on tools it never had written into it ahead of time. The bridge turns the discovered MCP tools into OpenAI's expected function-calling shape; the agent loop sends them to the model and runs whatever the model picks, routing the call back through the MCP client. **If not:** you'd be back to hardcoding a `@tool` function for every single capability — exactly the thing MCP exists to avoid.

`src/tool_bridge.py`

Story: these are two small but essential adapters between MCP's shapes and OpenAI's. `mcp_tools_to_openai_schema` turns each discovered tool into the dict OpenAI's `tools` parameter expects — an MCP tool's `input_schema` is already valid JSON Schema, so this is mostly a reshaping copy, not a rewrite. `extract_text` pulls the readable text out of a `CallToolResult`, which comes back as a list of content blocks, not a plain string. **If not:** the model call would reject the mismatched tool shape outright, and printing a result would crash on an object that isn't a string.

```
# Adapters between MCP's shapes and OpenAI's. Kept in one file so the loop
# never has to know either library's exact types.
from src.utils.logger import get_logger


def one_openai_tool(namespaced_name, description, input_schema):
    return {
        "type": "function",
        "function": {
            "name": namespaced_name,
            "description": description,
            "parameters": input_schema,
        },
    }


def mcp_tools_to_openai_schema(named_tools):
    logger = get_logger(__name__)
```

```
    schemas = []
    for namespaced_name, tool in named_tools:
        description = tool.description
        if description is None:
            description = ""
        schemas.append(
            one_openai_tool(namespaced_name, description, tool.input_schema)
        )
    logger.info(f"converted {len(schemas)} MCP tools to OpenAI schema")
    return schemas


def extract_text(call_result):
    parts = []
    for block in call_result.content:
        text = getattr(block, "text", None)
        if text is not None:
            parts.append(text)
    return "\n".join(parts)
```

`named_tools` is the list of `(namespaced_name, mcp_tool)` pairs the pool already built in Step 1 — the bridge never touches a raw server, only what was discovered.

`src/server_pool.py` — two methods added

Story: the connection pool from Step 1 now also hands out the OpenAI-shaped tool list and dispatches a call by its namespaced `servername__toolname`. The agent loop calls these two methods and nothing else from the pool. **If not:** the agent would need to know about raw sessions and server routing itself, instead of the simpler "here are the tools, call this one by name."

Add these two methods to the `MCPServerPool` class from Step 1:

```
    def openai_tools(self):
        return mcp_tools_to_openai_schema(self.named_tools)

    async def call_tool(self, namespaced_name, arguments):
        server_name, tool_name = self.routing[namespaced_name]
        session = self.sessions[server_name]
        result = await session.call_tool(tool_name, arguments)
        return extract_text(result)
```

and the imports at the top of the file:

```
from src.tool_bridge import extract_text, mcp_tools_to_openai_schema
```

(Step 4 wraps `call_tool`'s dispatch in a `try/except`; for now a clean call is enough.)

`src/models/llm.py`

Story: this is the one file that talks to the OpenAI SDK directly — it creates the client, and one `ask_model()` function that sends the conversation plus the discovered tools and returns the model's message. **If not:** OpenAI calls would be scattered through the agent loop, and swapping the model or adding a startup key check would mean editing several files.

```
# The only file that talks to the OpenAI SDK: the client, a startup key
# check, and the one tool-calling request the ReAct loop makes.
import openai
from openai import OpenAI

MODEL = "gpt-4o-mini"

def create_client(config):
    return OpenAI(api_key=config.openai_api_key)

def check_api_key_at_startup(client):
    try:
        client.chat.completions.create(
            model=MODEL,
            messages=[{"role": "user", "content": "ping"}],
            max_tokens=1,
        )
    except openai.AuthenticationError:
        return False
    return True

def ask_model(client, messages, tools):
    response = client.chat.completions.create(
        model=MODEL, messages=messages, tools=tools
    )
    return response.choices[0].message
```

`src/agent.py`

Story: this is the hand-built ReAct loop (no `create_agent` / `AgentExecutor` from a library) — ask the model, run any tool calls it requests through the pool, append each result as a `tool` message,

and repeat until the model answers with no further tool calls. A plain `max_iterations` cap stops a runaway loop from going forever. **If not:** a confused model could call tools indefinitely, and the "discovered tools" idea would have no actual loop to prove it works end to end.

```
# The ReAct loop: think (ask the model) -> act (call a discovered tool via
# the pool) -> observe (feed the result back). Bounded by max_iterations.
import json
from dataclasses import dataclass

from src.exceptions import MaxIterationsExceeded
from src.models.llm import ask_model
from src.utils.logger import get_logger


@dataclass
class AgentResult:
    answer: str
    steps: list


def build_messages(task):
    system = (
        "You are a helpful agent. Use the available tools to answer the "
        "task. When you have enough information, answer in plain words."
    )
    return [
        {"role": "system", "content": system},
        {"role": "user", "content": task},
    ]


async def run_agent(task, pool, client, max_iterations=6):
    logger = get_logger(__name__)
    messages = build_messages(task)
    tools = pool.openai_tools()
    steps = []

    for step_number in range(max_iterations):
        reply = ask_model(client, messages, tools)
        if not reply.tool_calls:
            logger.info("agent finished with a final answer")
            return AgentResult(answer=reply.content, steps=steps)

        messages.append({
            "role": "assistant",
            "content": reply.content,
            "tool_calls": _tool_calls_as_dicts(reply.tool_calls),
```

```
        })
        for call in reply.tool_calls:
            name = call.function.name
            arguments = json.loads(call.function.arguments)
            logger.info(f"step {step_number}: calling {name} {arguments}")
            observation = await pool.call_tool(name, arguments)
            steps.append({"tool": name, "args": arguments,
                          "observation": observation})
            messages.append({
                "role": "tool",
                "tool_call_id": call.id,
                "content": observation,
            })

    raise MaxIterationsExceeded(
        f"agent did not finish within {max_iterations} steps"
    )


def _tool_calls_as_dicts(tool_calls):
    out = []
    for call in tool_calls:
        out.append({
            "id": call.id,
            "type": "function",
            "function": {
                "name": call.function.name,
                "arguments": call.function.arguments,
            },
        })
    return out
```

`main.py` — now runs the agent

```
# Step 2 demo: connect to one server, discover its tools, and run the
# agent on a real task — printing each tool call and the final answer.
import asyncio
import sys

from src.agent import run_agent
from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import create_client
from src.server_pool import MCPServerPool
from src.servers_config import SERVERS
```

```
TASK = "What does my note about the Q3 roadmap say?"


async def run_demo():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    client = create_client(config)
    pool = MCPServerPool(SERVERS, connect_timeout=config.connect_timeout)
    await pool.connect_all()
    print(f"Connected servers: {list(pool.sessions.keys())}")
    print(f"Tools discovered: {len(pool.named_tools)}")
    try:
        print(f"\nTask: {TASK}")
        result = await run_agent(
            TASK, pool, client, max_iterations=config.max_iterations
        )
        for step in result.steps:
            print(f"  -> {step['tool']} {step['args']}")
        print(f"Answer: {result.answer}")
    finally:
        await pool.aclose()


def main():
    asyncio.run(run_demo())


if __name__ == "__main__":
    main()
```

Expected output (`python main.py` , with a real key) — the chained case: `search_notes` finds the note id, then `read_note` reads it using that id.

```
Connected servers: ['notes']
Tools discovered: 2

Task: What does my note about the Q3 roadmap say?
  -> notes__search_notes {'query': 'roadmap'}
  -> notes__read_note {'note_id': 'n1'}
Answer: Your Q3 roadmap note says to ship the billing API first, then the
mobile app beta.
```

tests/unit/test_tool_bridge.py

```
# Checks the schema bridge with tiny fakes — no server, no network.
# Run from the repo root: python -m tests.unit.test_tool_bridge
from src.tool_bridge import extract_text, mcp_tools_to_openai_schema


class FakeTool:
    def __init__(self, name, description, input_schema):
        self.name = name
        self.description = description
        self.input_schema = input_schema


class FakeBlock:
    def __init__(self, text):
        self.text = text


class FakeResult:
    def __init__(self, texts):
        self.content = []
        for text in texts:
            self.content.append(FakeBlock(text))


def test_schema_shape():
    tool = FakeTool("search_notes", "Search notes.",
                    {"type": "object", "properties": {}})
    schemas = mcp_tools_to_openai_schema([("notes__search_notes", tool)])
    one = schemas[0]
    result = "OK"
    if one["type"] != "function":
        result = "WRONG"
    if one["function"]["name"] != "notes__search_notes":
        result = "WRONG"
    if one["function"]["description"] != "Search notes.":
        result = "WRONG"
    if "properties" not in one["function"]["parameters"]:
        result = "WRONG"
    print(f"case 1 (openai schema shape)   -> {result}")


def test_missing_description():
    tool = FakeTool("t", None, {"type": "object"})
    schemas = mcp_tools_to_openai_schema([("s__t", tool)])
```

```
    result = "OK"
    if schemas[0]["function"]["description"] != "":
        result = "WRONG"
    print(f"case 2 (None description -> '') -> {result}")


def test_extract_text_joins():
    result_obj = FakeResult(["line one", "line two"])
    text = extract_text(result_obj)
    result = "OK"
    if text != "line one\nline two":
        result = "WRONG"
    print(f"case 3 (extract_text joins)    -> {result}")


if __name__ == "__main__":
    test_schema_shape()
    test_missing_description()
    test_extract_text_joins()
```

Expected output:

```
case 1 (openai schema shape)   -> OK
case 2 (None description -> '') -> OK
case 3 (extract_text joins)    -> OK
```

`tests/integration/test_agent.py` (cases 1–2)

Story: this is the real proof — launch the example server over a genuine MCP connection and confirm the agent actually discovers its tools and chains two real tool calls in one run. **If not:** a unit test built on fakes could pass while the real protocol path underneath it was silently broken.

```
# Integration test: launches the real example servers over MCP. Each case
# prints OK or WRONG. Run from the repo root:
#   python -m tests.integration.test_agent
import asyncio

from src.agent import run_agent
from src.config import load_config
from src.models.llm import create_client
from src.server_pool import MCPServerPool

NOTES = {"name": "notes", "command": "python",
         "args": ["servers/notes_server.py"]}
```

```
async def connect(servers):
    config = load_config()
    client = create_client(config)
    pool = MCPServerPool(servers, connect_timeout=config.connect_timeout)
    await pool.connect_all()
    return config, client, pool


async def test_discovery_namespaced():
    config, client, pool = await connect([NOTES])
    names = []
    for namespaced, tool in pool.named_tools:
        names.append(namespaced)
    result = "OK"
    if "notes__search_notes" not in names:
        result = "WRONG"
    if "notes__read_note" not in names:
        result = "WRONG"
    await pool.aclose()
    print(f"case 1 (tools discovered + namespaced) -> {result}")


async def test_chained_single_server():
    config, client, pool = await connect([NOTES])
    out = await run_agent("What does my note about the Q3 roadmap say?",
                          pool, client, config.max_iterations)
    result = "OK"
    if len(out.steps) != 2:
        result = "WRONG"
    if out.steps[0]["tool"] != "notes__search_notes":
        result = "WRONG"
    if out.steps[1]["tool"] != "notes__read_note":
        result = "WRONG"
    await pool.aclose()
    print(f"case 2 (chained search then read)      -> {result}")


async def main():
    await test_discovery_namespaced()
    await test_chained_single_server()


if __name__ == "__main__":
    asyncio.run(main())
```

Expected output (cases 3–4 are added in Steps 3 and 4):

```
case 1 (tools discovered + namespaced) -> OK
case 2 (chained search then read)      -> OK
```